# Документация модуля загрузки файлов

Функции для поиска и скачивания .raw файлов с веб-страниц

Версия 1.0

17 февраля 2026 г.

## Содержание


**1 Общее описание** **2**

1.1 Основные возможности . . . . . 2

**2 Функции для скачивания файлов** **2**

2.1 download_file . . . . . 2

2.2 download_files_batch . . . . . 3

2.3 resume_download . . . . . 4

**3 Функции для поиска файлов** **5**

3.1 find_raw_files_on_page . . . . . 5

3.2 get_file_info . . . . . 6

3.3 print_files_info . . . . . 7

**4 Специализированные функции** **7**

4.1 find_and_download_13_raw . . . . . 7

4.2 interactive_file_selector . . . . . 8

**5 Константы и настройки** **8**

**6 Обработка ошибок** **9**

**7 Примеры использования** **9**

7.1 Базовый пример . . . . . 9

7.2 Расширенный пример с мониторингом . . . . . 10

7.3 Интеграция с основным модулем обработки данных . . . . . 11

**8 Рекомендации по использованию** **12**

**9 Зависимости** **13**

# 1 Общее описание

Модуль предоставляет набор функций для поиска и скачивания файлов с веб-страниц. Специализирован для работы с .raw файлами, имеющими определенный паттерн в имени (например, `*.13.raw`), но может быть легко адаптирован для любых типов файлов.

## 1.1 Основные возможности

- Поиск файлов на веб-странице по регулярным выражениям
- Получение информации о файлах без скачивания (размер, тип, дата модификации)
- Скачивание файлов с отображением прогресса
- Пакетное скачивание нескольких файлов
- Интерактивный выбор файлов для скачивания
- Поддержка докачки прерванных файлов
- Автоматическое переименование при совпадении имен
- Обработка ошибок и таймаутов

# 2 Функции для скачивания файлов

## 2.1 download_file

```python
def download_file(url: str, download_dir: str = "downloads",
                  filename: Optional[str] = None,
                  chunk_size: int = 8192, timeout: int = 30,
                  verify: bool = True) -> Optional[str]:
```

**Назначение:** Основная функция для скачивания файла с отображением прогресса.
**Параметры:**

**url: str** – URL файла для скачивания

**download_dir: str** – директория для сохранения (по умолчанию: "downloads")

**filename: Optional[str ]** – имя файла (если None, берется из URL)

**chunk_size: int** – размер чанка для скачивания в байтах (по умолчанию: 8192)

**timeout: int** – таймаут соединения в секундах (по умолчанию: 30)

**verify: bool** – проверять SSL сертификат (по умолчанию: True)

**Возвращаемое значение:**

- `Optional[str]` – путь к сохраненному файлу или None при ошибке

**Особенности:**

- Автоматически создает директорию для скачивания, если она не существует
- При совпадении имени добавляет числовой суффикс (например, `file_1.raw`)
- Отображает прогресс-бар с процентом, скоростью и размером
- Обрабатывает различные типы ошибок (таймаут, соединение, HTTP ошибки)

**Пример использования:**

```python
# Простое скачивание
filepath = download_file("http://example.com/data/file.13.raw")

# Скачивание с пользовательским именем
filepath = download_file(
    "http://example.com/data/file.13.raw",
    download_dir="my_data",
    filename="measurement_1.raw"
)

# Скачивание с увеличенным таймаутом для больших файлов
filepath = download_file(
    "http://example.com/data/large_file.raw",
    timeout=120,
    chunk_size=16384
)
```

**Пример вывода:**

```
Скачивание: file.13.raw
Из: http://example.com/data/file.13.raw
В: downloads/file.13.raw
Прогресс: || 100.0% (1024.0/1024.0 KB) [512.3 KB/s]
 Скачивание завершено за 2.1с
  Размер: 1024.0 KB
```

## 2.2 download_files_batch

```python
def download_files_batch(urls: List[str], download_dir: str = "downloads",
                         max_files: int = None, delay: float = 0.5) -> List[str]:
```

**Назначение:** Пакетное скачивание нескольких файлов последовательно.
**Параметры:**

**urls: List[str ]** – список URL для скачивания

**download_dir: str** – директория для сохранения

**max_files: int** – максимальное количество файлов для скачивания

**delay: float** – задержка между запросами в секундах

**Возвращаемое значение:**

- `List[str]` – список успешно скачанных файлов

**Пример использования:**

```python
# Скачивание всех найденных файлов
urls = [
    "http://example.com/data/file1.13.raw",
    "http://example.com/data/file2.13.raw",
    "http://example.com/data/file3.13.raw"
]
downloaded = download_files_batch(urls, download_dir="batch_downloads")

# Скачивание первых 5 файлов с задержкой 1 секунда
downloaded = download_files_batch(urls, max_files=5, delay=1.0)
```

**Пример вывода:**

```
Начинаю скачивание 3 файлов...
============================================================

[1/3] Обработка файла...
Скачивание: file1.13.raw
Прогресс: || 100.0% (512.0/512.0 KB) [512.3 KB/s]
 Скачивание завершено

[2/3] Обработка файла...
...

============================================================
Скачано 3 из 3 файлов
```

## 2.3 resume_download

```python
def resume_download(url: str, download_dir: str = "downloads",
                    chunk_size: int = 8192) -> Optional[str]:
```

**Назначение:** Скачивание файла с поддержкой докачки (если файл уже частично скачан).

**Параметры:**

**url: str** – URL файла

**download_dir: str** – директория для сохранения

**chunk_size: int** – размер чанка

**Возвращаемое значение:**

- `Optional[str]` – путь к файлу или None

**Особенности:**

- Проверяет наличие частично скачанного файла
- Использует HTTP заголовок `Range` для докачки
- Проверяет поддержку докачки сервером
- Автоматически переключается на обычное скачивание, если сервер не поддерживает докачку

**Пример использования:**

```
# Прерванное скачивание можно продолжить
filepath = resume_download(
    "http://example.com/data/large_file.raw",
    download_dir="downloads"
)

# Если файл уже частично скачан, докачка начнется с места остановки
```

**Пример вывода:**

```
Найден частично скачанный файл: 5242880 байт
Скачивание: large_file.raw
Всего размер: 10485760.0 KB
Прогресс: 75.5% (7912.5/10485.8 KB) [512.3 KB/s]
```

# 3 Функции для поиска файлов

## 3.1 find_raw_files_on_page

```
def find_raw_files_on_page(url: str, pattern: str = r'\.13\.raw$') -> List[str]:
```

**Назначение:** Поиск файлов на веб-странице по заданному паттерну.
**Параметры:**

**url: str** – URL страницы для поиска

**pattern: str** – регулярное выражение для фильтрации файлов

**Возвращаемое значение:**

- `List[str]` – список найденных URL файлов

**Пример использования:**

```
# Поиск всех .13.raw файлов
files = find_raw_files_on_page("http://example.com/data/")

# Поиск с другим паттерном
```

```python
files = find_raw_files_on_page(
    "http://example.com/data/",
    pattern=r'\.14\.raw$'
)

# Поиск всех .raw файлов
files = find_raw_files_on_page(
    "http://example.com/data/",
    pattern=r'\.raw$'
)
```

## 3.2 get_file_info

```python
def get_file_info(file_url: str) -> Optional[dict]:
```

**Назначение:** Получение информации о файле без скачивания (HEAD запрос).
**Параметры:**

**file_url: str** – URL файла

**Возвращаемое значение:**

- `Optional[dict]` – словарь с информацией о файле:

  **url** – URL файла

  **filename** – имя файла

  **size** – размер в байтах

  **size_str** – размер в человеко-читаемом формате

  **type** – MIME тип файла

  **last_modified** – дата последней модификации

**Пример использования:**

```python
info = get_file_info("http://example.com/data/file.13.raw")
if info:
    print(f"Файл: {info['filename']}")
    print(f"Размер: {info['size_str']}")
    print(f"Тип: {info['type']}")
    print(f"Изменен: {info['last_modified']}")
```

**Пример вывода:**

```
Файл: file.13.raw
Размер: 1.5 MB
Тип: application/octet-stream
Изменен: Wed, 15 Feb 2024 10:30:45 GMT
```

### 3.3 print_files_info

```python
def print_files_info(files: Union[List[str], List[dict]],
                     title: str = "Найденные файлы") -> None:
```

**Назначение:** Форматированный вывод информации о файлах в виде таблицы.
**Параметры:**

**files** – список URL или список словарей с информацией

**title** – заголовок для вывода

**Пример вывода:**

```
Найденные файлы:
--------------------------------------------------------------------------------------------
#   Имя файла                                   Размер       Дата модификации            Ти
--------------------------------------------------------------------------------------------
1   measurement_1.13.raw                        1.5 MB       Wed, 15 Feb 2024 10:30:45   ap
2   measurement_2.13.raw                         512 KB       Wed, 15 Feb 2024 11:20:12   a
3   calibration.13.raw                           2.1 MB       Tue, 14 Feb 2024 09:15:33   a
```

# 4 Специализированные функции

### 4.1 find_and_download_13_raw

```python
def find_and_download_13_raw(url: str, download: bool = True,
                             max_files: int = None) -> List[str]:
```

**Назначение:** Комбинированная функция для поиска и скачивания файлов `.13.raw`.
**Параметры:**

**url: str** – URL страницы для поиска

**download: bool** – скачивать ли найденные файлы

**max_files: int** – максимальное количество файлов для скачивания

**Возвращаемое значение:**

- `List[str]` – список найденных файлов

**Пример использования:**

```python
# Только поиск
files = find_and_download_13_raw("http://example.com/data/", download=False)

# Поиск и скачивание всех файлов
files = find_and_download_13_raw("http://example.com/data/", download=True)

# Поиск и скачивание первых 10 файлов
files = find_and_download_13_raw("http://example.com/data/",
                                 download=True, max_files=10)
```

## 4.2 interactive_file_selector

```python
def interactive_file_selector(url: str, pattern: str = r'\.13\.raw$') -> None:
```

**Назначение:** Интерактивный выбор файлов для скачивания через консоль.
**Параметры:**

**url: str** – URL страницы

**pattern: str** – паттерн для фильтрации файлов

**Возможности выбора:**

- Ввод номеров через запятую (например: `1,3,5`)
- Ввод диапазона (например: `1-10`)
- Команда `all` для выбора всех файлов
- Команда `q` для выхода

**Пример использования:**

```python
interactive_file_selector("http://example.com/data/")
```

**Пример интерактивной сессии:**

```
Выберите файлы для скачивания:
  - Введите номера через запятую (например: 1,3,5)
  - Введите диапазон (например: 1-10)
  - Введите 'all' для скачивания всех
  - Введите 'q' для выхода

Ваш выбор: 1-3

Выбрано 3 файлов
Начинаю скачивание 3 файлов...
```

# 5 Константы и настройки

Таблица 1: Параметры конфигурации

| Параметр | Значение по умолчанию | Описание |
|---|---|---|
| `chunk_size` | 8192 | Размер чанка для скачивания (байт) |
| `timeout` | 30 | Таймаут соединения (секунд) |

| Параметр | Значение по умолчанию | Описание |
|---|---|---|
| `delay` | 0.5 | Задержка между запросами (секунд) |
| `verify` | True | Проверка SSL сертификата |
| `download_dir` | "downloads" | Директория по умолчанию |

# 6 Обработка ошибок

Модуль включает комплексную обработку различных типов ошибок:

Таблица 2: Типы ошибок и их обработка

| Тип ошибки | Причина | Действие |
|---|---|---|
| `Timeout` | Превышение времени ожидания | Вывод сообщения, возврат None |
| `ConnectionError` | Проблемы с соединением | Вывод сообщения, возврат None |
| `HTTPError` | HTTP ошибка (4xx, 5xx) | Вывод кода ошибки, возврат None |
| `FileNotFoundError` | Не найдена директория | Создание директории |
| `OSError` | Проблемы с файловой системой | Вывод сообщения, возврат None |

**Пример обработки ошибок:**

```
try:
    filepath = download_file("http://example.com/data/file.raw")
    if filepath:
        print(f"Файл сохранен: {filepath}")
    else:
        print("Не удалось скачать файл")
except Exception as e:
    print(f"Неожиданная ошибка: {e}")
```

# 7 Примеры использования

## 7.1 Базовый пример

```
import requests
from pathlib import Path

# Настройка
url = "http://example.com/data/"
```

```python
download_dir = "measurements"

# Создание директории
Path(download_dir).mkdir(exist_ok=True)

# Поиск файлов
print("Поиск файлов...")
files = find_raw_files_on_page(url, pattern=r'\.13\.raw$')

if files:
    print(f"Найдено {len(files)} файлов")

    # Получение информации
    for file_url in files[:3]:  # Первые 3 файла
        info = get_file_info(file_url)
        if info:
            print(f"  {info['filename']} - {info['size_str']}")

    # Скачивание
    answer = input("\nСкачать все файлы? (y/n): ")
    if answer.lower() == 'y':
        downloaded = download_files_batch(files, download_dir)
        print(f"Скачано {len(downloaded)} файлов")
else:
    print("Файлы не найдены")
```

## 7.2 Расширенный пример с мониторингом

```python
def monitor_new_files(url: str, check_interval: int = 3600):
    """
    Мониторинг появления новых файлов.

    Args:
        url: URL страницы
        check_interval: интервал проверки в секундах
    """
    import time
    from datetime import datetime

    downloaded = set()

    print(f"Начат мониторинг {url}")
    print(f"Интервал проверки: {check_interval}с")

    while True:
        try:
            print(f"\n[{datetime.now()}] Проверка...")

            # Поиск файлов
            files = find_raw_files_on_page(url, pattern=r'\.13\.raw$')
```

```python

            # Поиск новых файлов
            new_files = [f for f in files if f not in downloaded]

            if new_files:
                print(f"Найдено {len(new_files)} новых файлов")

                # Скачивание новых файлов
                for file_url in new_files:
                    info = get_file_info(file_url)
                    if info:
                        print(f"  Новый файл: {info['filename']} ({info['size_str']})")

                    # Скачивание
                    downloaded_file = download_file(
                        file_url,
                        download_dir="auto_downloads",
                        timeout=60
                    )

                    if downloaded_file:
                        downloaded.add(file_url)

                    time.sleep(1)  # Задержка между файлами
            else:
                print("Новых файлов не найдено")

            # Ожидание до следующей проверки
            print(f"Ожидание {check_interval}с...")
            time.sleep(check_interval)

        except KeyboardInterrupt:
            print("\nМониторинг остановлен")
            break
        except Exception as e:
            print(f"Ошибка: {e}")
            time.sleep(60)  # Ожидание минуту при ошибке

# Запуск мониторинга
# monitor_new_files("http://example.com/data/", check_interval=3600)
```

## 7.3 Интеграция с основным модулем обработки данных

```python
# Импорт модулей
from raw_data_parser import parse_data_file, get_channel_statistics
from file_downloader import find_and_download_13_raw

def download_and_analyze(url: str, channel: int = 0):
    """
    Скачивание и анализ данных.
```

```python
    """
    # Скачивание файлов
    files = find_and_download_13_raw(url, download=True, max_files=5)

    if not files:
        print("Нет файлов для анализа")
        return

    # Анализ каждого скачанного файла
    for filepath in files:
        print(f"\nАнализ файла: {filepath}")
        print("-" * 40)

        try:
            # Парсинг
            events = parse_data_file(filepath)
            print(f"Записей в файле: {len(events)}")

            # Статистика по каналу
            stats = get_channel_statistics(events, channel)

            # Вывод статистики
            total = sum(stats.values())
            print(f"Всего срабатываний канала {channel}: {total}")

            if total > 0:
                print("Топ-5 активных ячеек:")
                sorted_cells = sorted(stats.items(), key=lambda x: x[1], reverse=True)
                for cell, count in sorted_cells[:5]:
                    percent = count / total * 100
                    print(f"  Ячейка {cell}: {count} раз ({percent:.1f}%)")

        except Exception as e:
            print(f"Ошибка при анализе {filepath}: {e}")

# Использование
download_and_analyze("http://example.com/data/", channel=0)
```

# 8 Рекомендации по использованию

1. **Для небольших файлов** (< 100 МБ) используйте `download_file()` с параметрами по умолчанию

2. **Для больших файлов** (> 100 МБ) увеличьте `timeout` и `chunk_size`:

```python
    download_file(url, timeout=300, chunk_size=65536)
```

3. **Для нестабильного соединения** используйте `resume_download()` для возможности докачки

4. **Для пакетного скачивания** установите разумную задержку (`delay`) чтобы не перегружать сервер
5. **Для автоматического мониторинга** используйте функцию с таймером и логированием
6. **Для работы через прокси** настройте сессию requests:

```
    proxies = {'http': 'http://proxy:8080', 'https': 'http://proxy:8080'}
    response = requests.get(url, proxies=proxies, timeout=30)
```

# 9 Зависимости

Для работы модуля требуются следующие библиотеки:

**requests** – для выполнения HTTP запросов

**re** – для работы с регулярными выражениями (стандартная библиотека)

**os, pathlib** – для работы с файловой системой (стандартная библиотека)

**time** – для задержек и измерения времени (стандартная библиотека)

**typing** – для аннотаций типов (стандартная библиотека)

**urllib.parse** – для работы с URL (стандартная библиотека)

Установка зависимостей:

```
pip install requests
```